

Engenharia de Software – PUC Campinas

Estrutura e Recuperação de Dados I

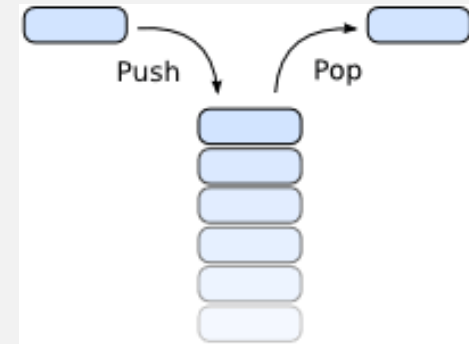
Prof. Dr. Paulo Cesar de Alvarenga Lucci

Pilhas (stack)

Pilhas

○ INTRODUÇÃO

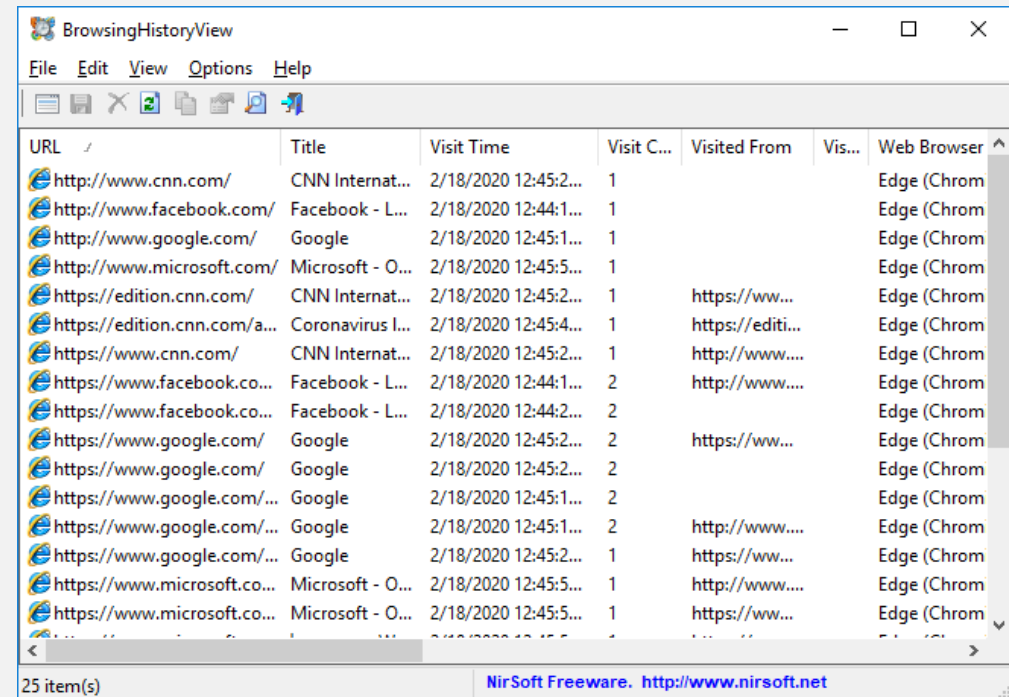
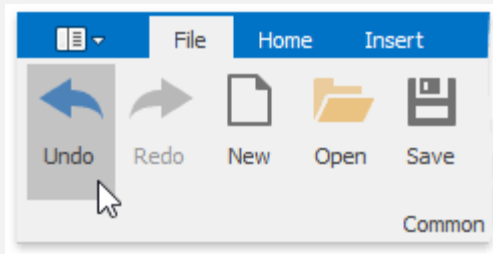
- Já vimos que listas encadeadas podem assumir comportamentos específicos.
- A fila é uma lista com a estratégia FIFO.
- Agora veremos outro caso particular: a **pilha**, que obedece à estratégia **LIFO**.
- **LIFO** – Last In, First Out ou seja: “o último que entra é o primeiro que sai”.
 - Inserção → sempre no topo (normalmente chamada de **push**).
 - Remoção → sempre do topo (normalmente chamada de **pop**).
- Analogia: uma pilha de pratos sujos.
 - O último inserido será o primeiro a ser lavado



Pilhas

○ APLICAÇÕES

- Históricos de navegação no browser.
- Mecanismo de Desfazer/Refazer em editores de texto.
- Controle de chamadas de funções (stack frame).



ESSAS APLICAÇÕES UTILIZAM ESTRUTURAS DE PILHAS!!!

Pilhas

○ PUSH E POP

- A função **push** insere um novo elemento no topo da pilha.
 - Parâmetros comuns:
 - ponteiro para a pilha e o valor a ser inserido.
 - Verificação importante: antes de inserir, deve verificar se a pilha está cheia (no caso de pilha com tamanho fixo).
 - Ação principal: incrementa o topo e insere o valor na nova posição do topo.
- A função **pop** remove e retorna o elemento do topo da pilha.
 - Parâmetros comuns:
 - ponteiro para a pilha.
 - Verificação importante: deve verificar se a pilha está vazia antes de remover.
 - Ação principal: retorna o valor no topo e decrementa o topo.

Pilhas

○ IMPLEMENTAÇÃO DE PILHA

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Node
5 typedef struct Node {
6     int value;
7     struct Node* next;
8 } Node;
9
10 // Stack
11 typedef struct Stack {
12     Node* top;
13 } Stack;
14
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
#include <stdio.h>
#include <stdlib.h>

// Node
typedef struct Node {
    int value;
    struct Node* next;
} Node;

// Stack
typedef struct Stack {
    Node* top;
} Stack;

// Create node
Node* create_node(int value) {
    Node* new_node = (Node*) malloc(sizeof(Node));
    new_node->value = value;
    new_node->next = NULL;
    return new_node;
}

// Push
Stack* push(Stack* s, int value) {
    Node* new_node = create_node(value);
    new_node->next = s->top;
    s->top = new_node;
    return s;
}

// Pop
Stack* pop(Stack* s) {
    if (s->top == NULL) return s;
    Node* temp = s->top;
    s->top = s->top->next;
    free(temp);
    return s;
}

// Free stack
void free_stack(Stack* s) {
    Node* current = s->top;
    while (current != NULL) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
    free(s);
}

// Print stack
void print_stack(Stack* s) {
    Node* current = s->top;
    printf("Stack (top->bottom): ");
    while (current != NULL) {
        printf("%d -> ", current->value);
        current = current->next;
    }
    printf("NULL\n");
}

// Example
int main() {
    Stack* s = (Stack*) malloc(sizeof(Stack));
    s->top = NULL;

    s = push(s, 10);
    s = push(s, 20);
    s = push(s, 30);
    print_stack(s);

    s = pop(s);
    print_stack(s);

    free_stack(s);
    return 0;
}
```

Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
30
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
39
40 // Free stack
41 void free_stack(Stack* s) {
42     Node* current = s->top;
43     while (current != NULL) {
44         Node* temp = current;
45         current = current->next;
46         free(temp);
47     }
48     free(s);
49 }
50
```

```
51 // Print stack
52 void print_stack(Stack* s) {
53     Node* current = s->top;
54     printf("Stack (top -> bottom): ");
55     while (current != NULL) {
56         printf("%d -> ", current->value);
57         current = current->next;
58     }
59     printf("NULL\n");
60 }
```

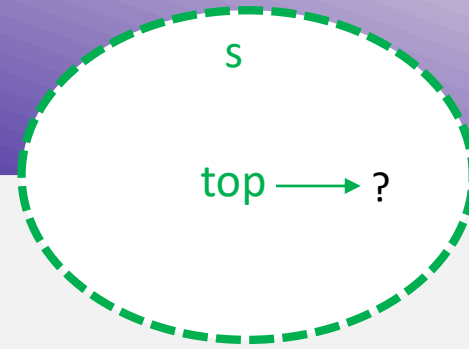
Exemplo de push

Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
10 // Stack
11 typedef struct Stack {
12     Node* top;
13 } Stack;
```



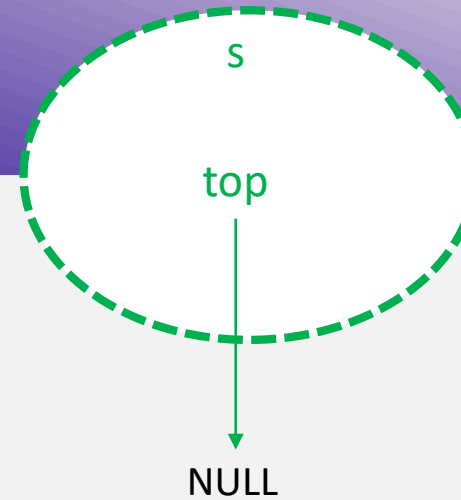
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



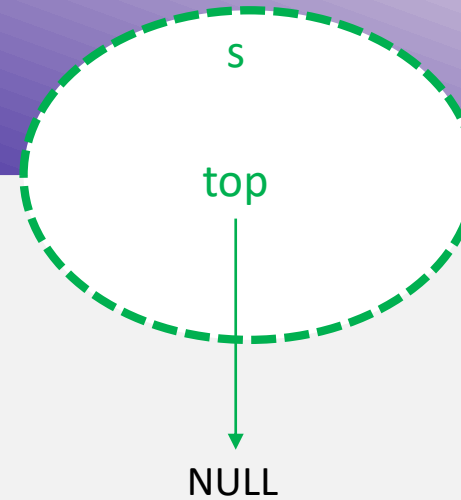
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66     s = push(s, 10);
67     s = push(s, 20);
68     s = push(s, 30);
69     print_stack(s);
70 }
71
72 free_stack(s);
73 return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



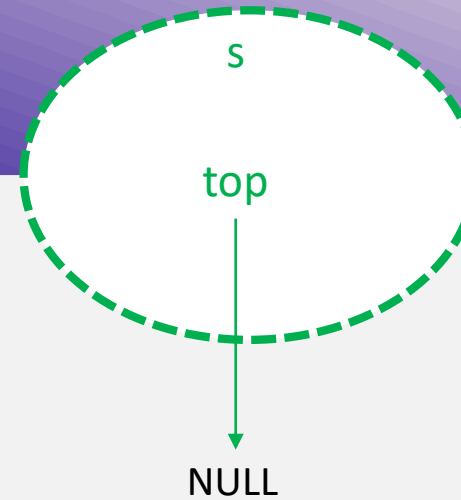
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66     s = push(s, 10);
67     s = push(s, 20);
68     s = push(s, 30);
69     print_stack(s);
70 }
71
72 free_stack(s);
73 return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



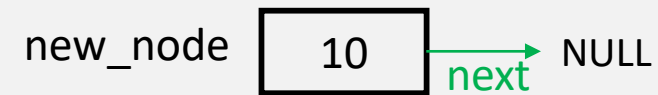
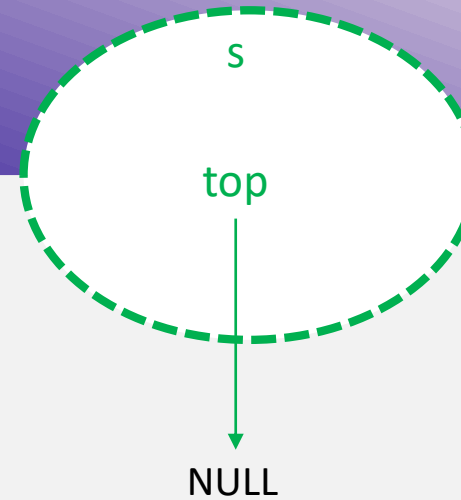
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66     s = push(s, 10);
67     s = push(s, 20);
68     s = push(s, 30);
69     print_stack(s);
70 }
71
72 free_stack(s);
73 return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



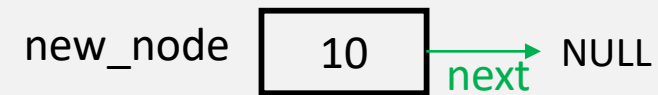
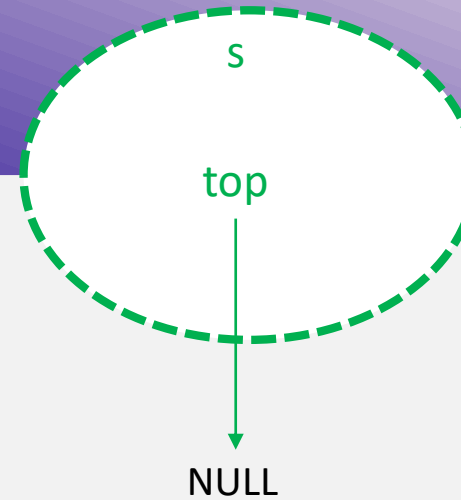
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66     s = push(s, 10);
67     s = push(s, 20);
68     s = push(s, 30);
69     print_stack(s);
70 }
71
72 free_stack(s);
73 return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



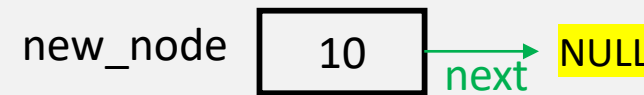
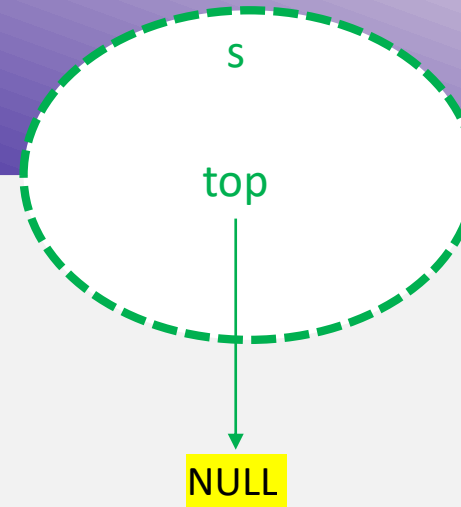
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66     s = push(s, 10);
67     s = push(s, 20);
68     s = push(s, 30);
69     print_stack(s);
70 }
71
72 free_stack(s);
73 return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



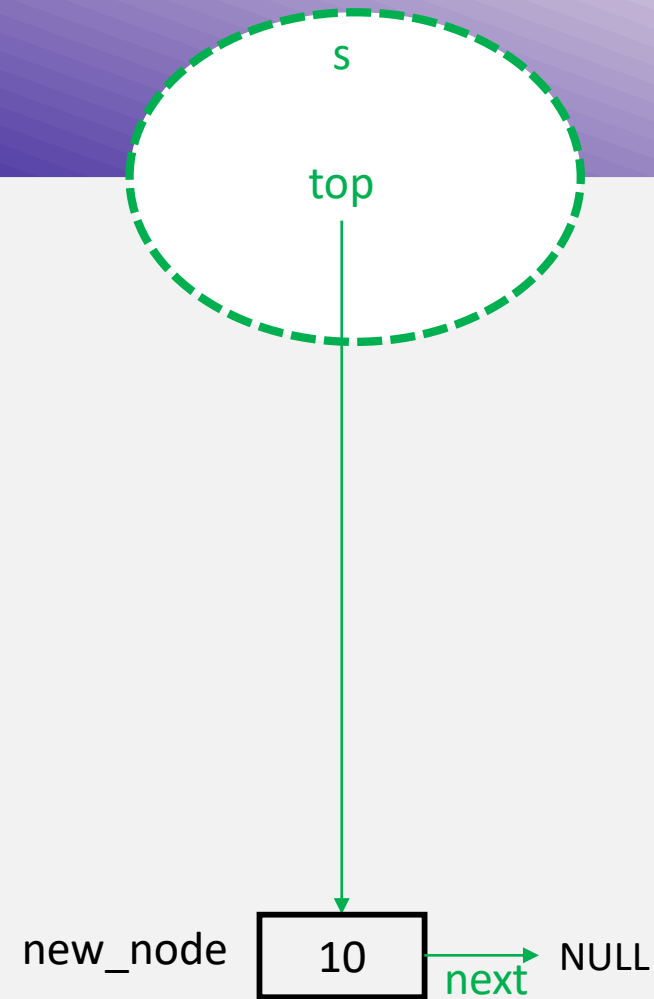
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66     s = push(s, 10);
67     s = push(s, 20);
68     s = push(s, 30);
69     print_stack(s);
70 }
71
72 free_stack(s);
73 return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



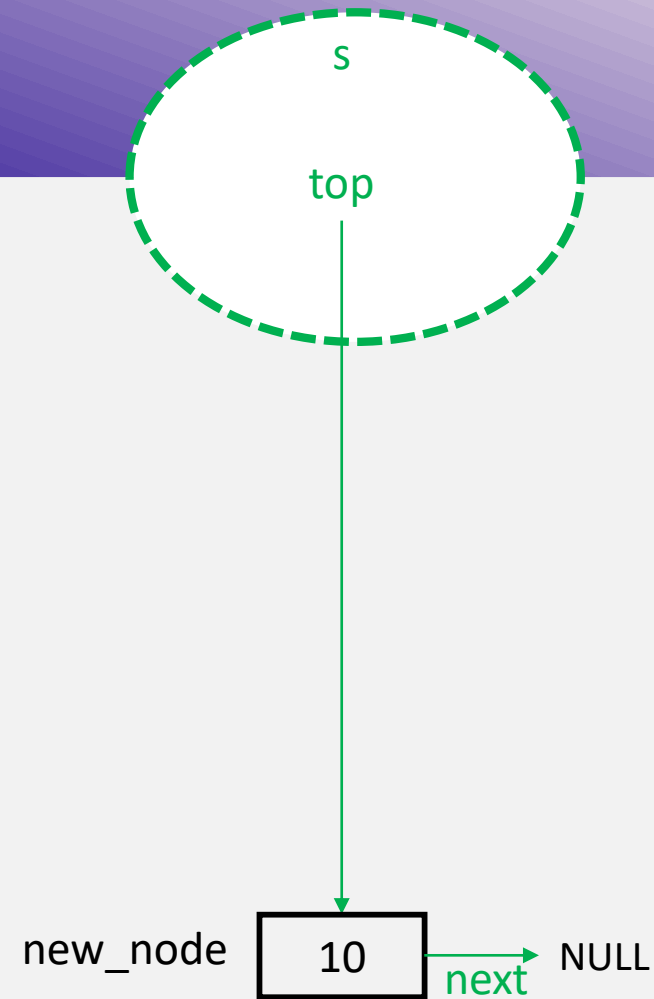
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66     s = push(s, 10);
67     s = push(s, 20);
68     s = push(s, 30);
69     print_stack(s);
70 }
71
72 free_stack(s);
73 return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



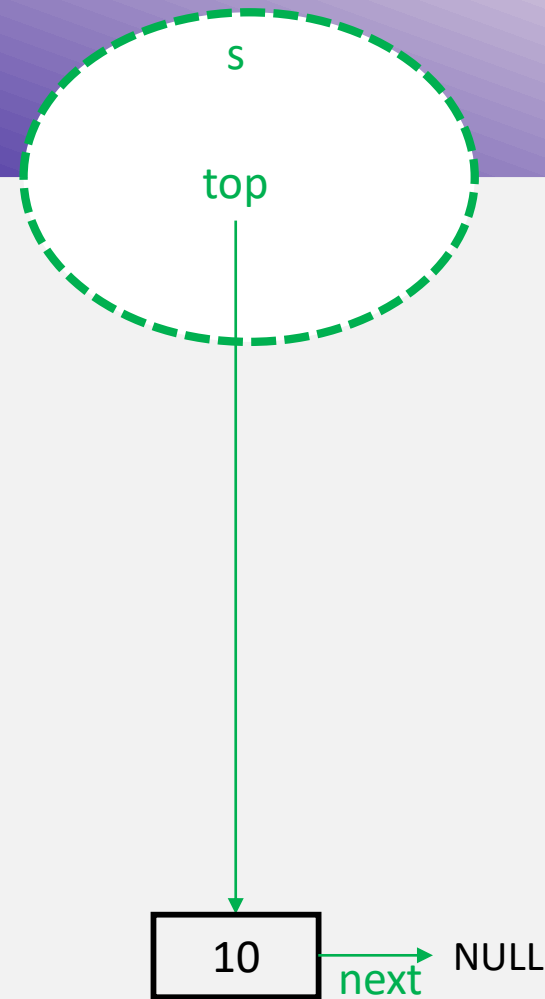
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66     s = push(s, 10);
67     s = push(s, 20);
68     s = push(s, 30);
69     print_stack(s);
70 }
71
72 free_stack(s);
73 return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



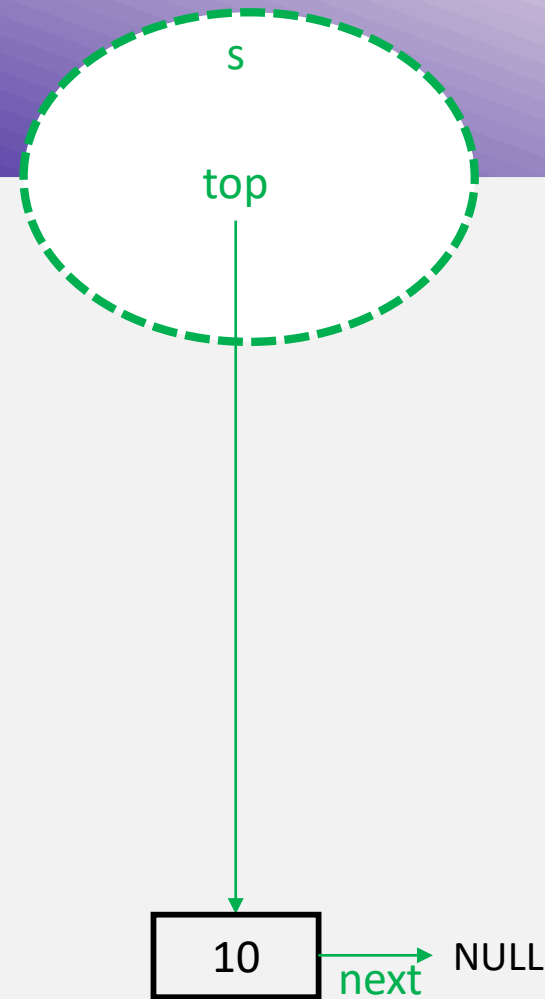
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



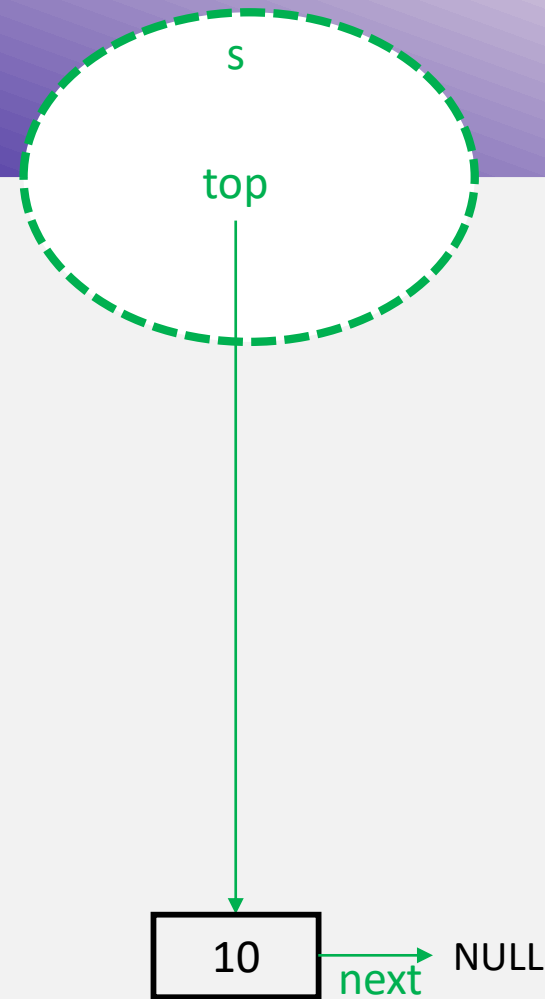
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



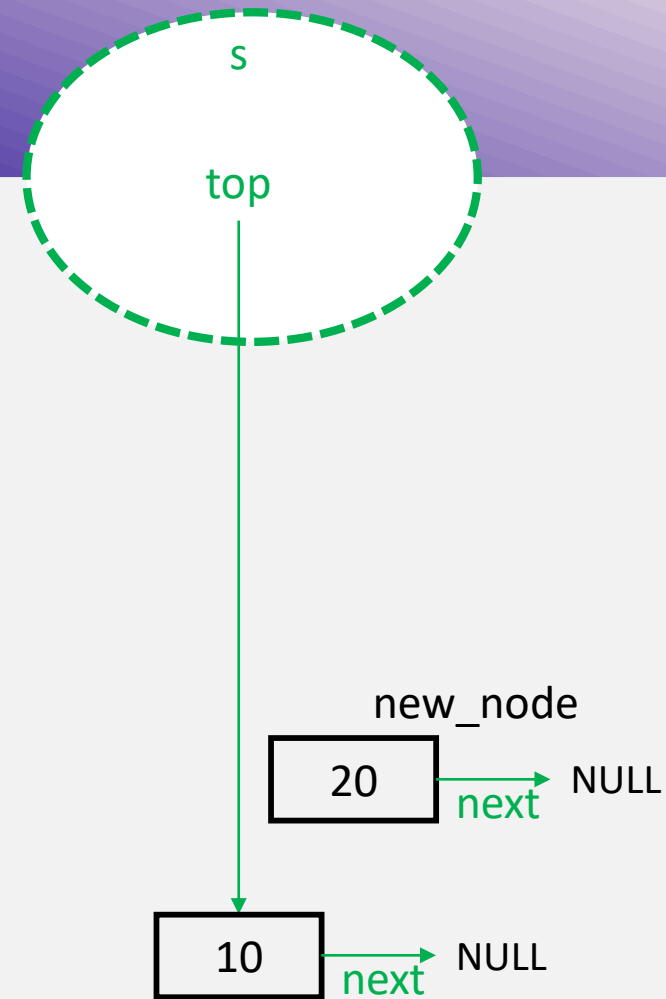
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



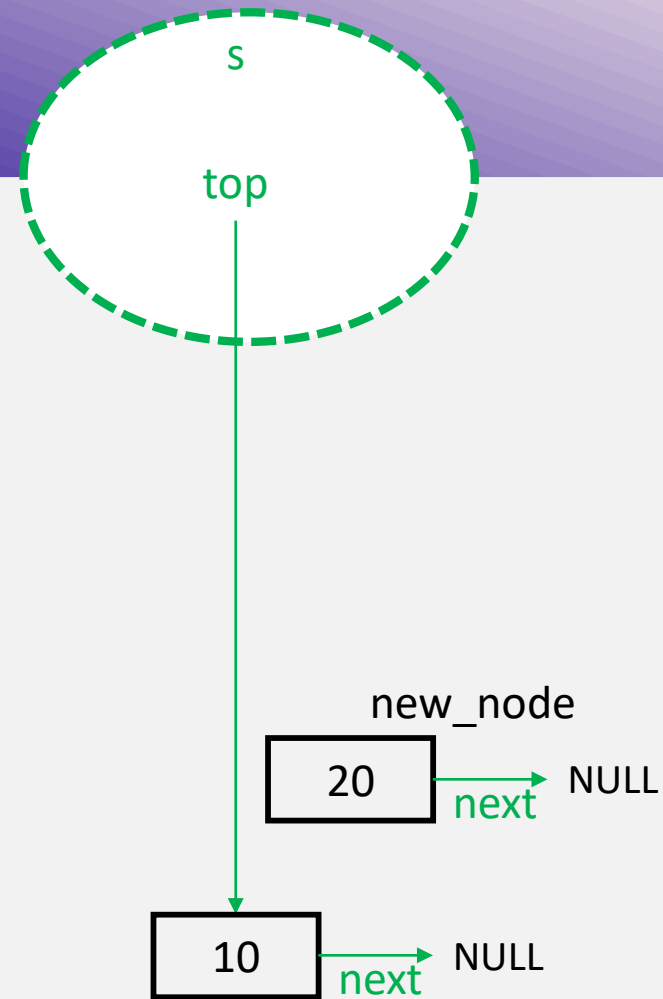
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



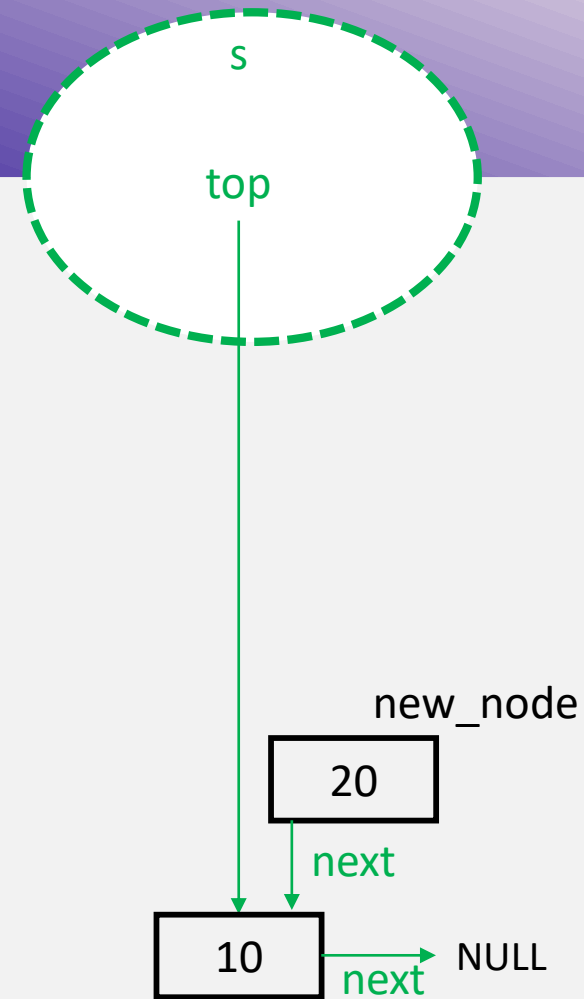
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



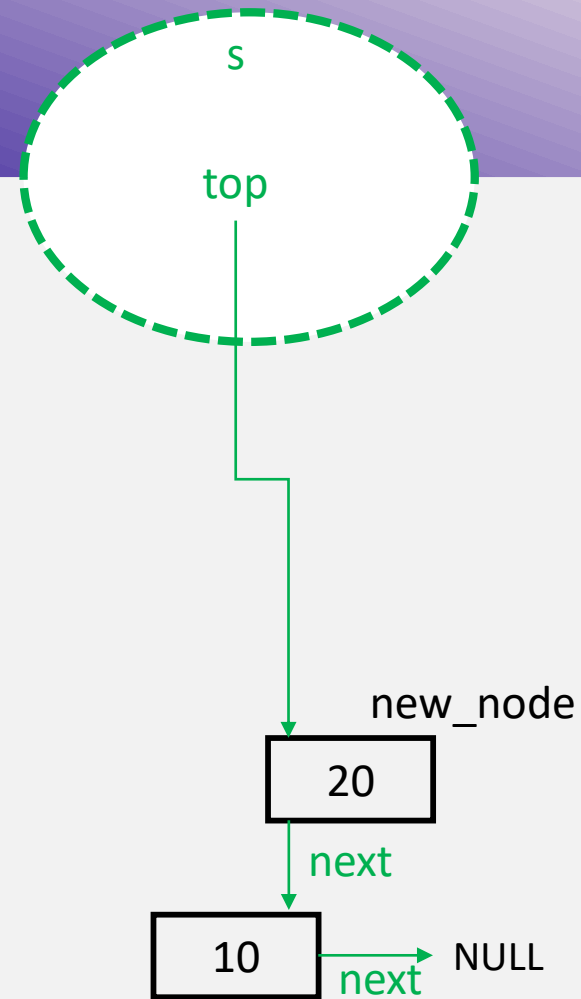
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



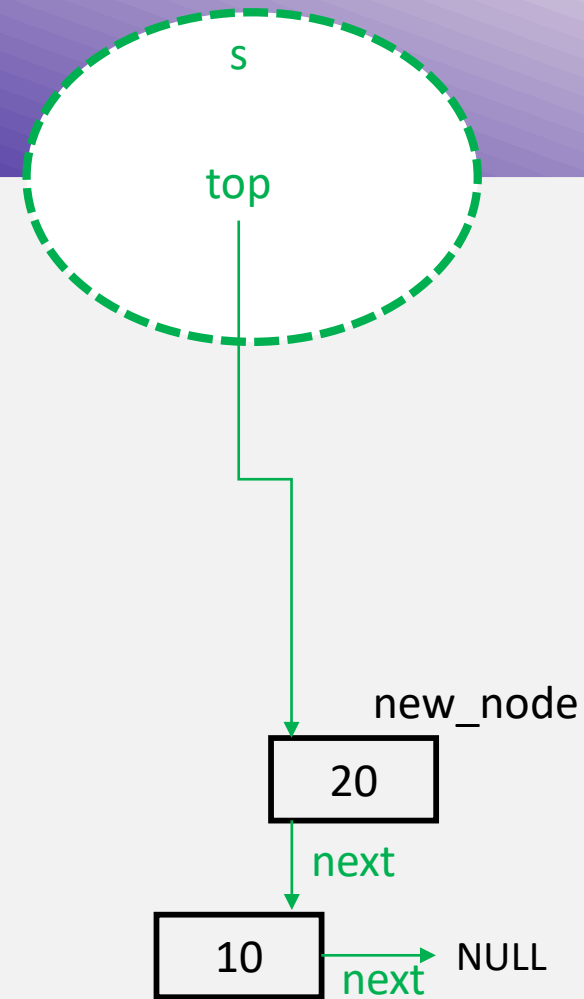
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



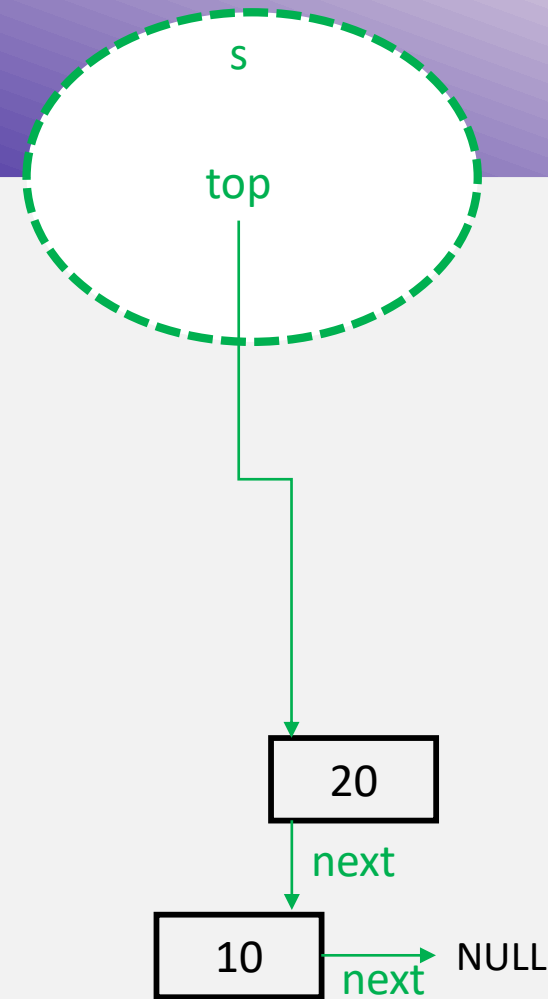
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



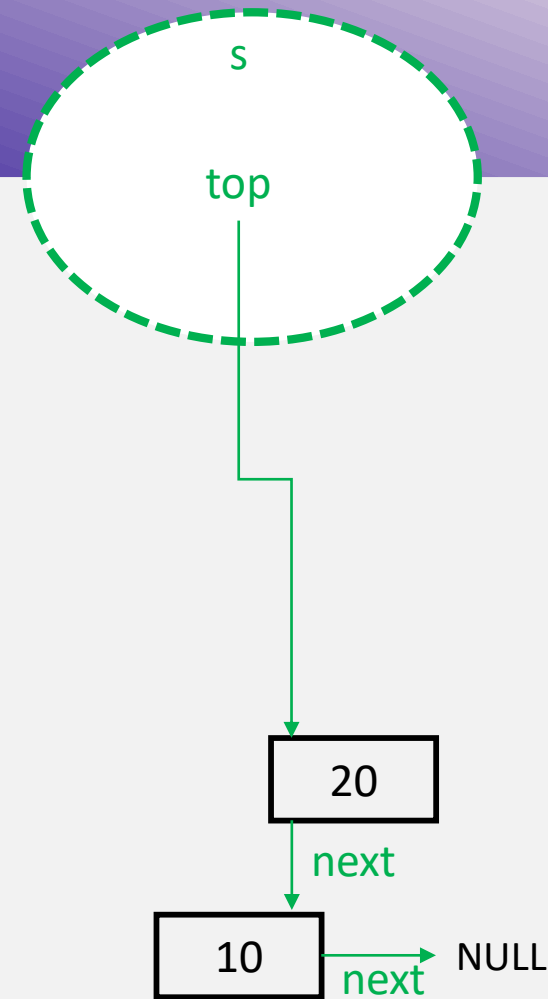
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



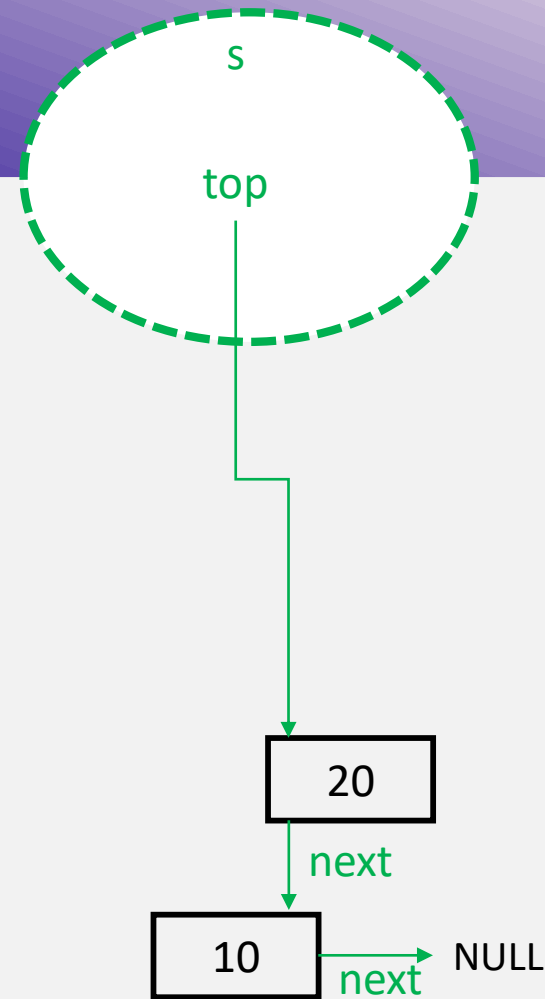
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



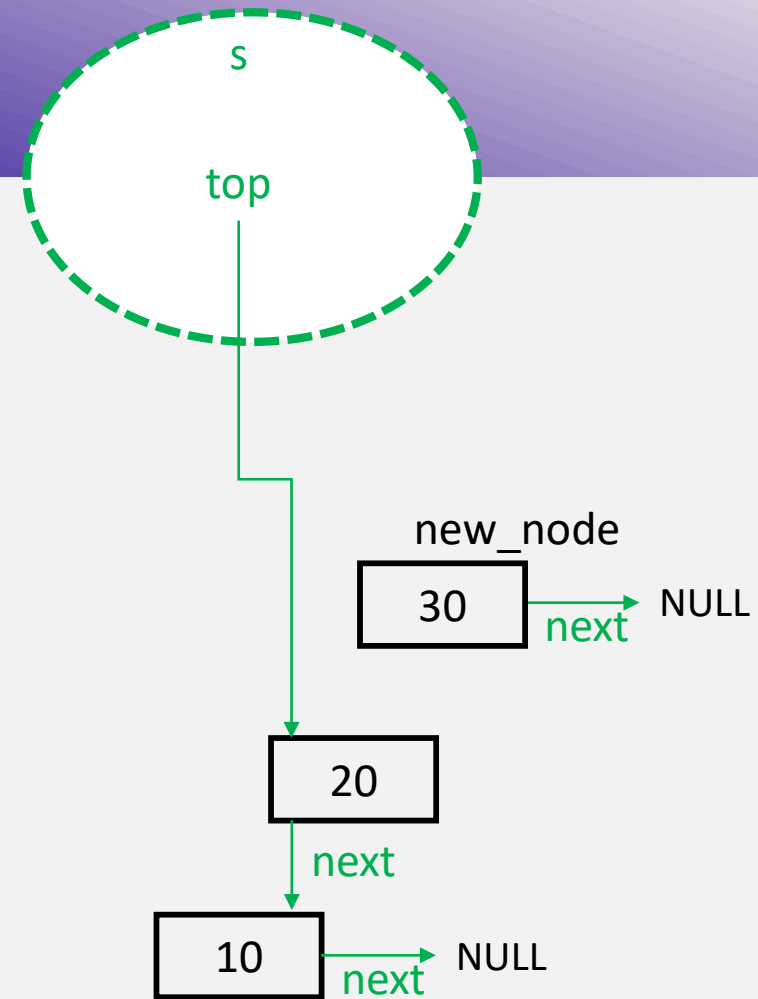
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



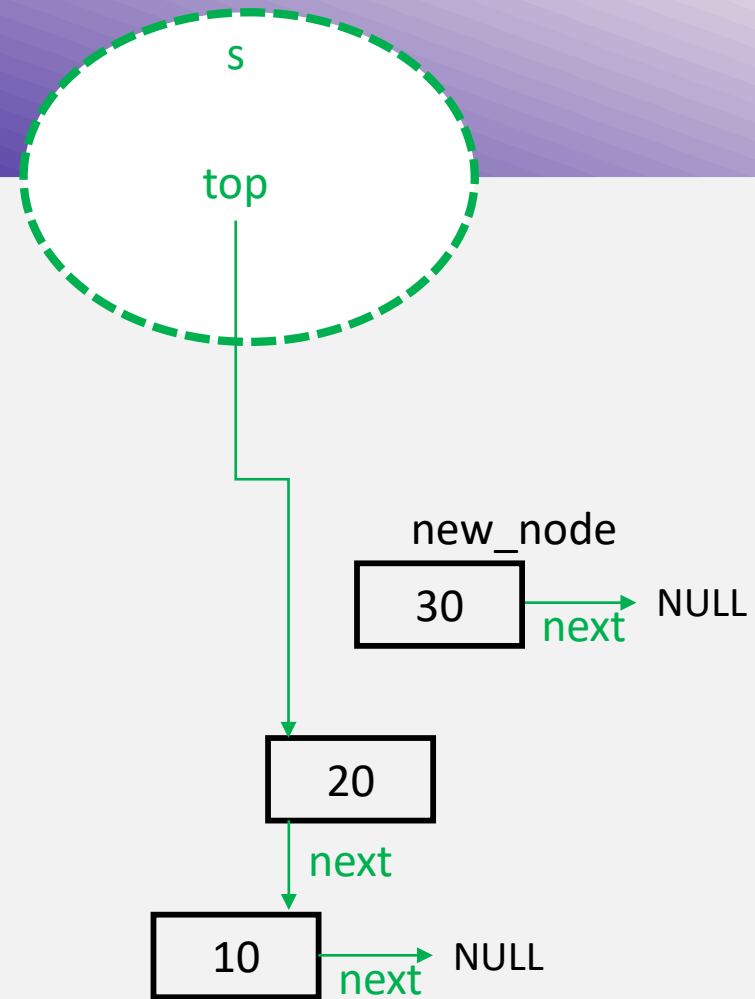
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



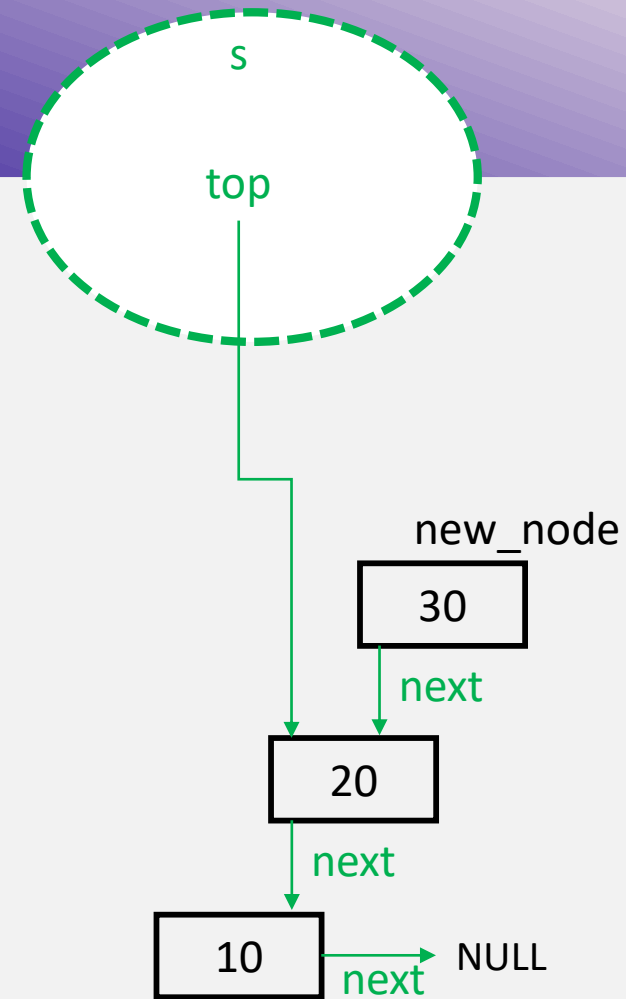
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



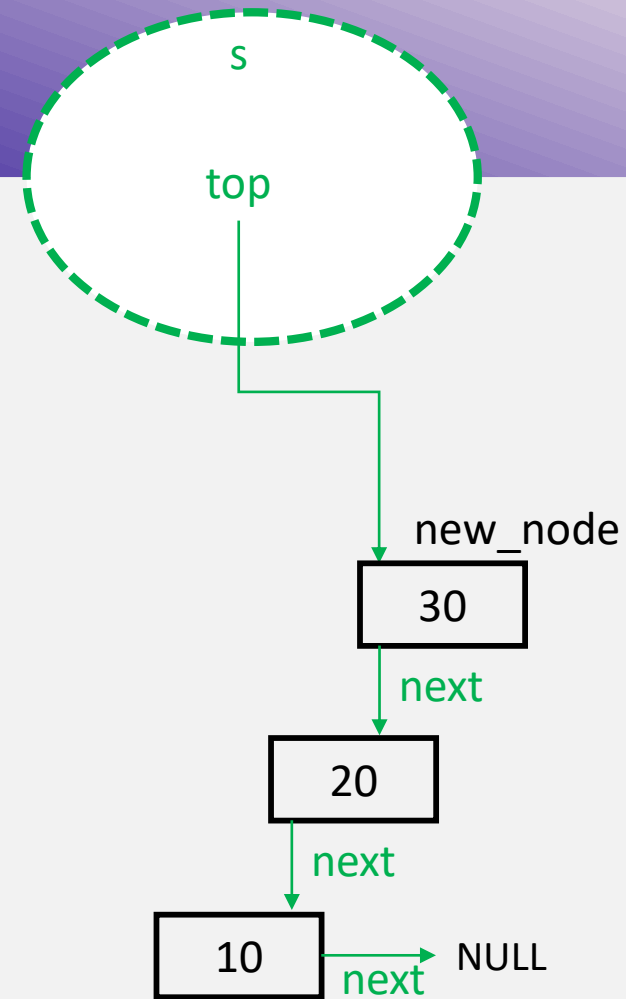
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



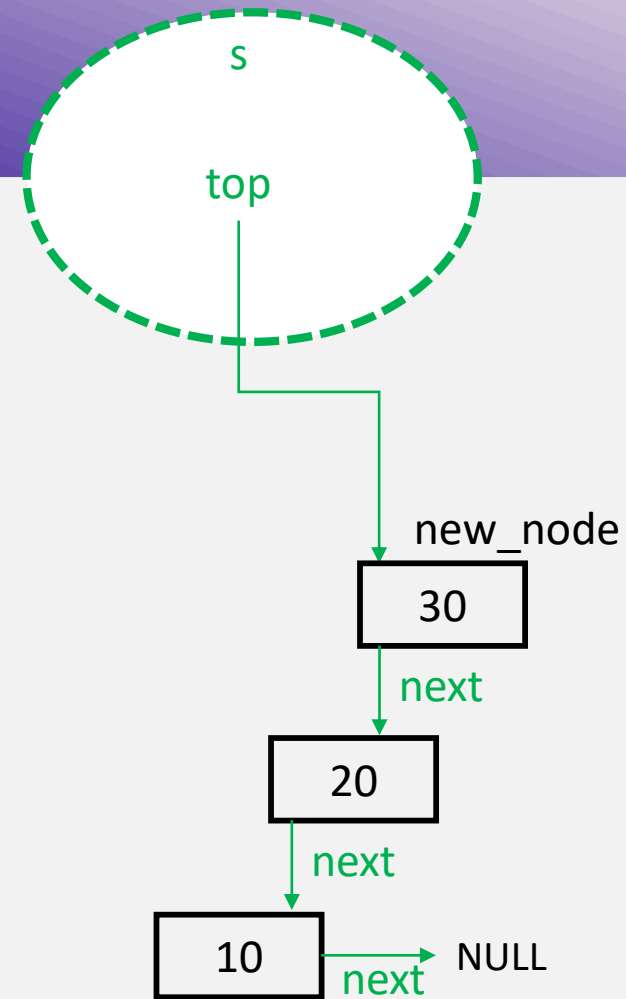
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



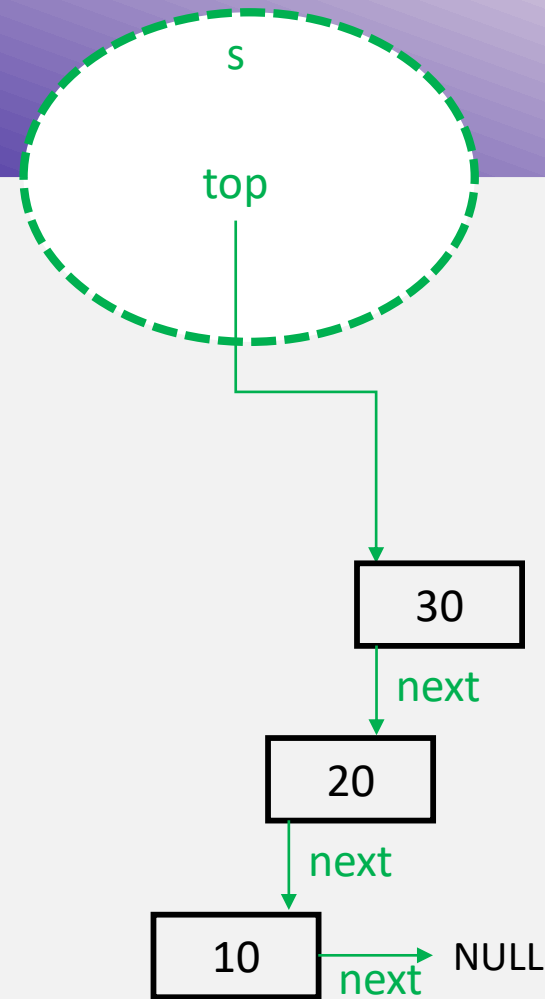
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



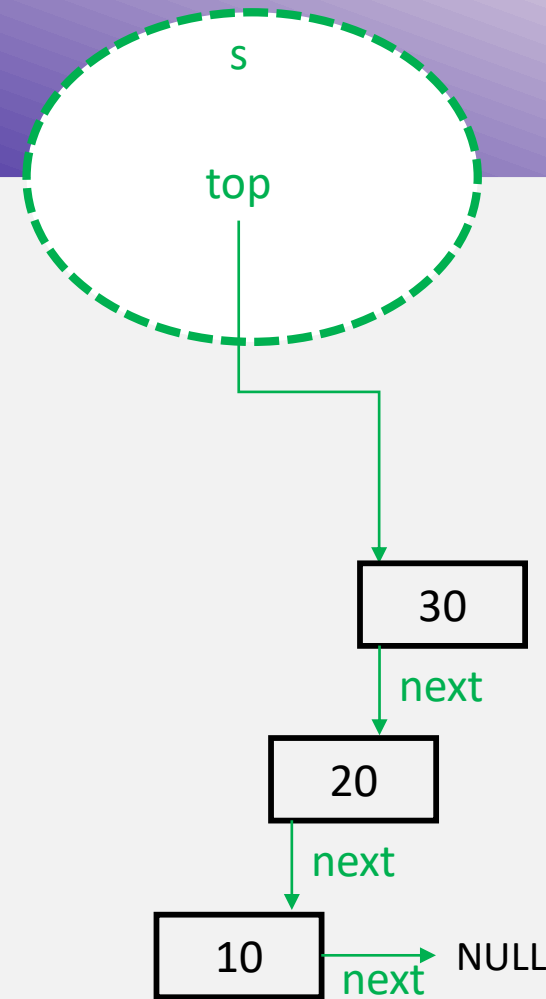
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



```
Stack (top -> bottom): 30 -> 20 -> 10 -> NULL
```

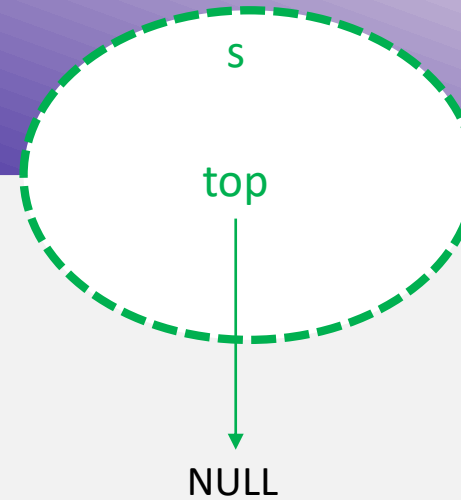
Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



```
Stack (top -> bottom): 30 -> 20 -> 10 -> NULL
```

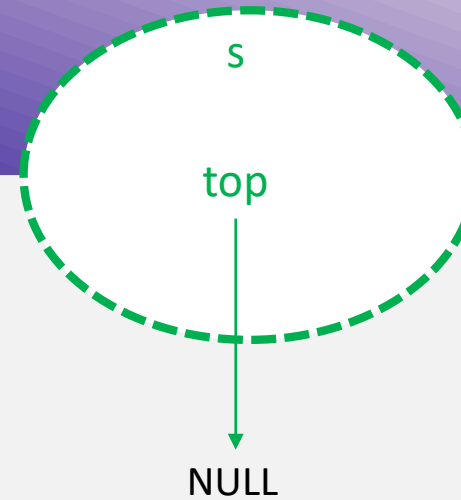
Pilhas

◦ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70     print_stack(s);
71
72     free_stack(s);
73     return 0;
74 }
```

```
15 // Create node
16 Node* create_node(int value) {
17     Node* new_node = (Node*) malloc(sizeof(Node));
18     new_node->value = value;
19     new_node->next = NULL;
20     return new_node;
21 }
```

```
23 // Push
24 Stack* push(Stack* s, int value) {
25     Node* new_node = create_node(value);
26     new_node->next = s->top;
27     s->top = new_node;
28     return s;
29 }
```



```
Stack (top -> bottom): 30 -> 20 -> 10 -> NULL

...Program finished with exit code 0
Press ENTER to exit console.
```

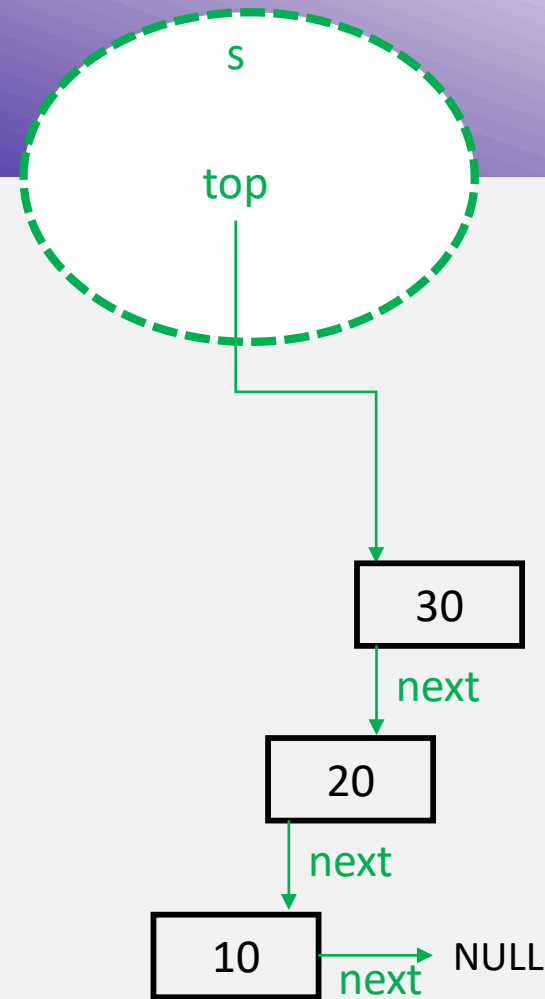
Exemplo de pop

Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

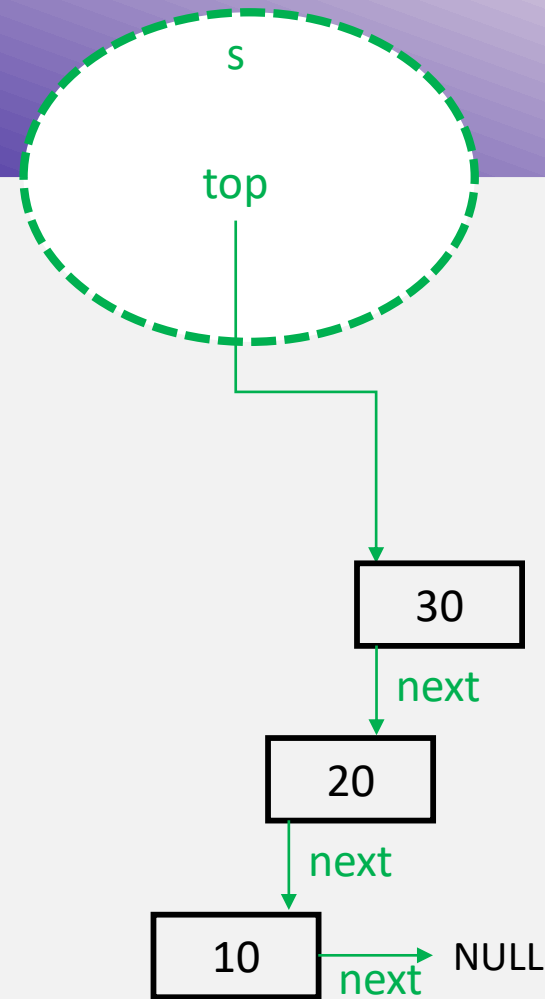


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

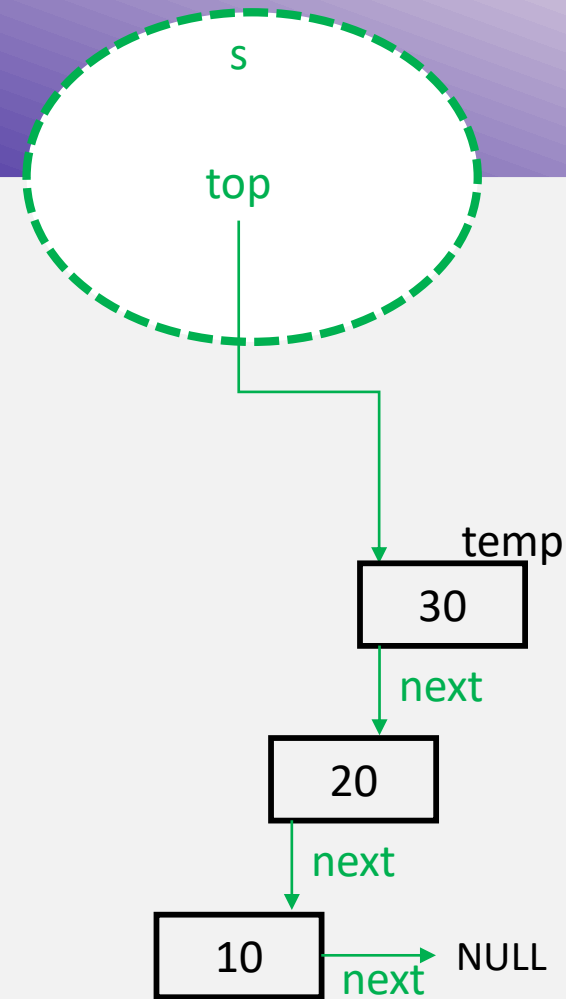


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

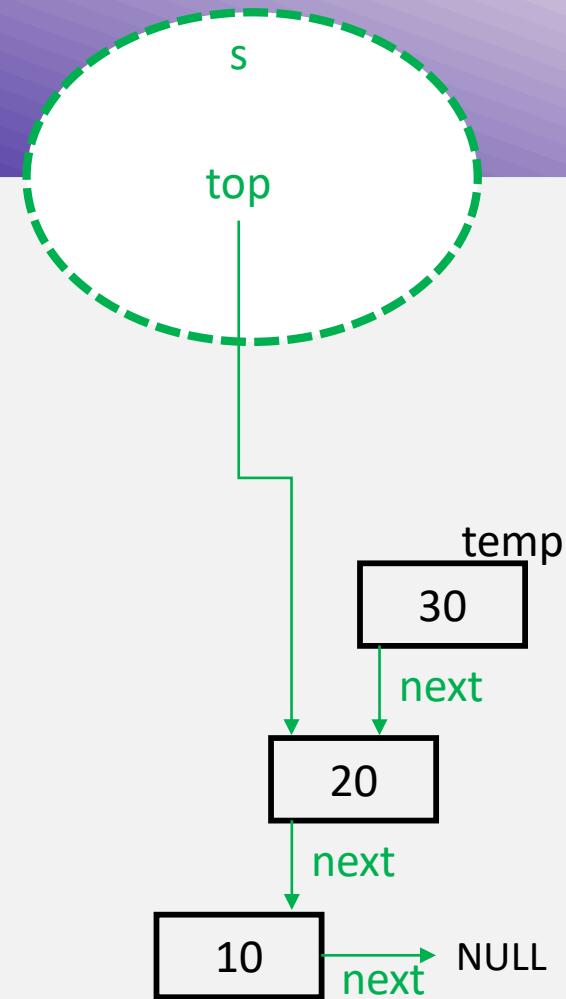


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

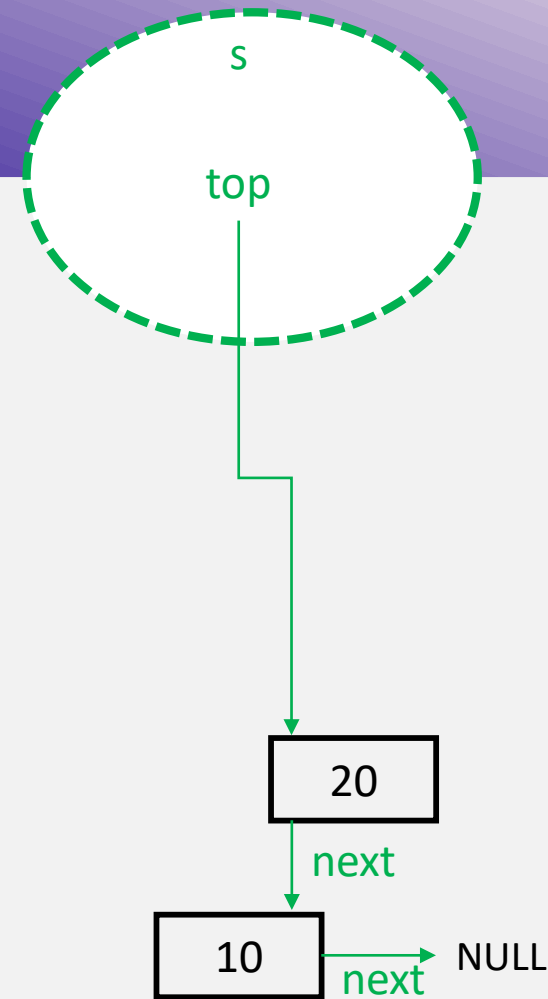


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

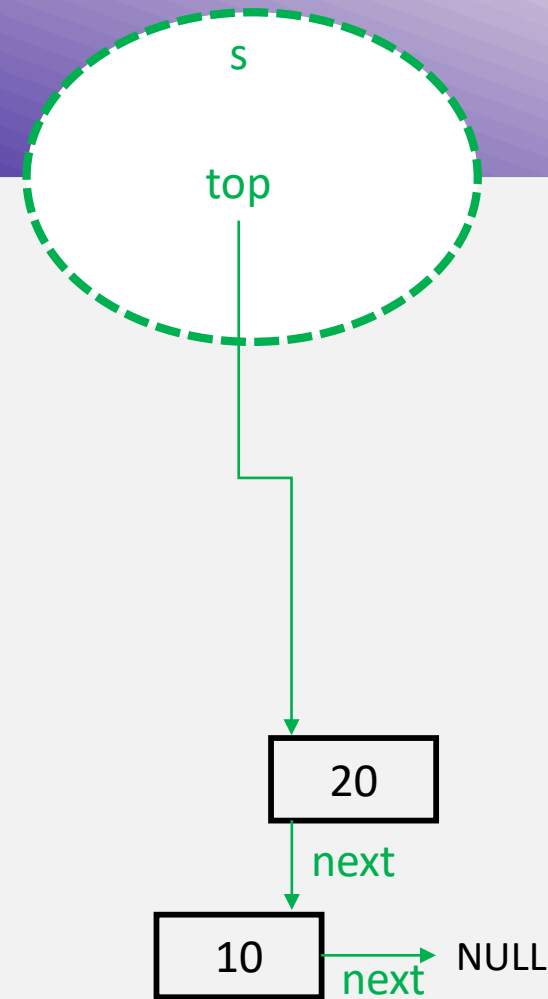


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

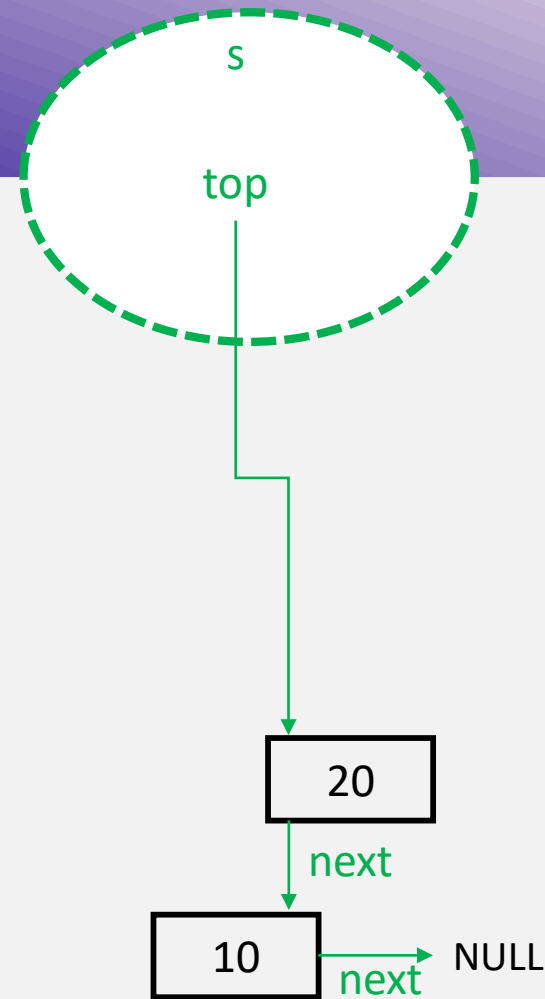


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

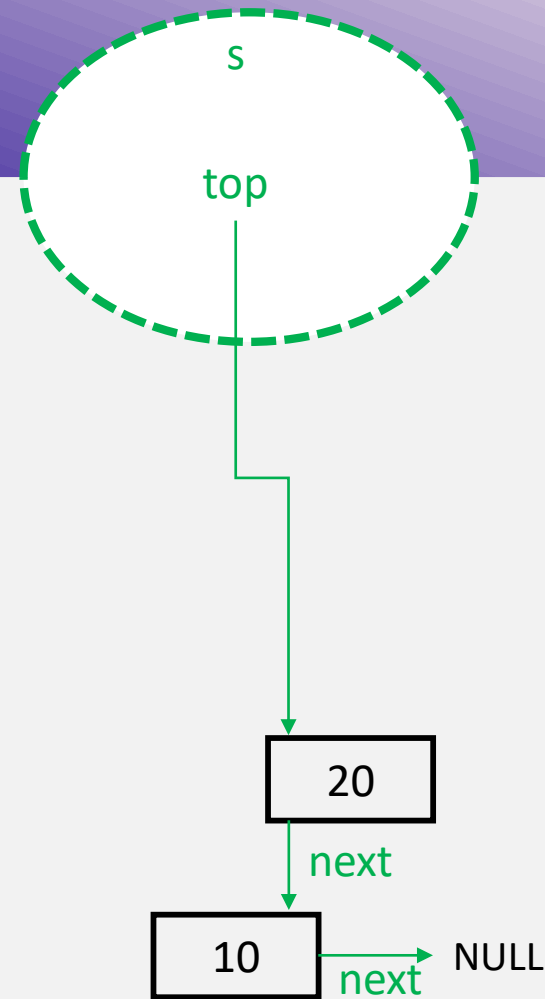


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

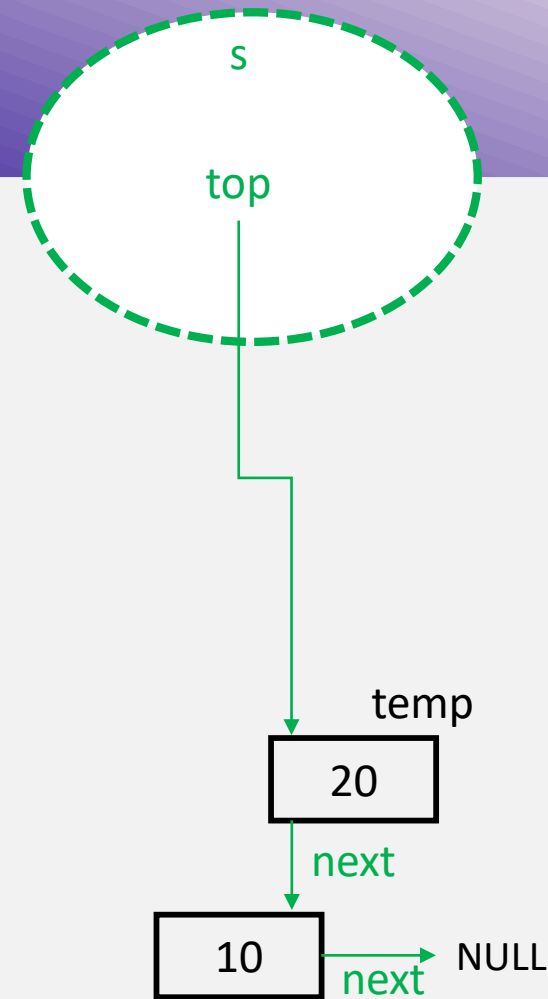


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

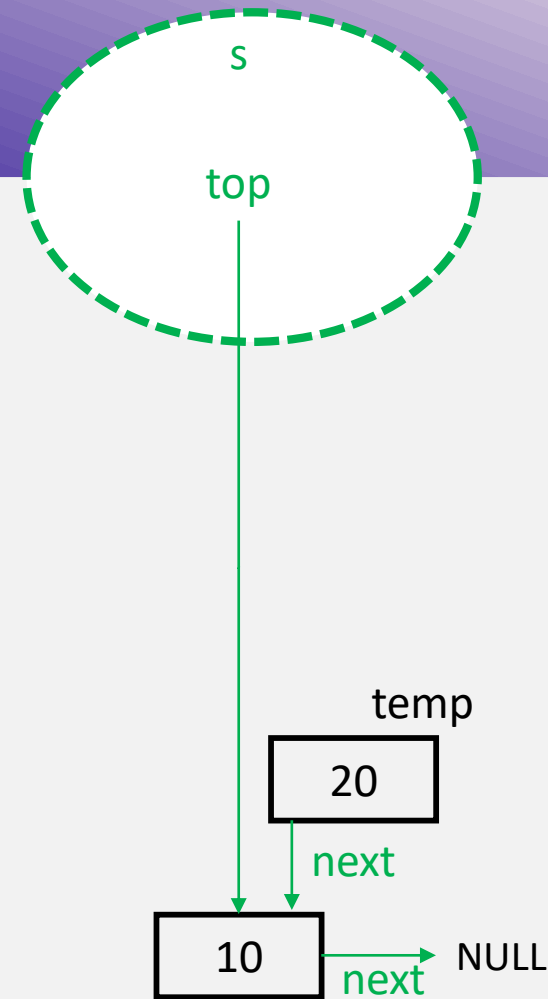


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

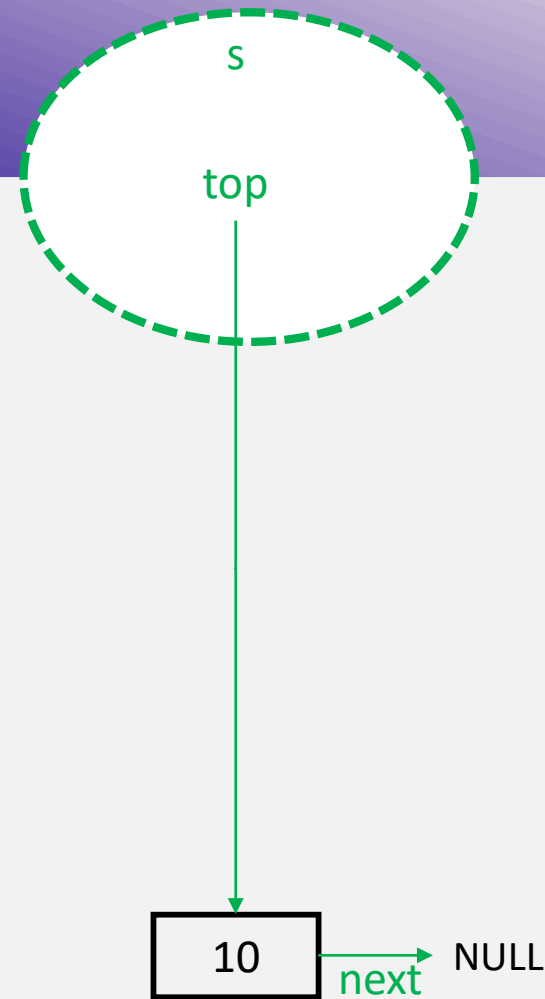


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

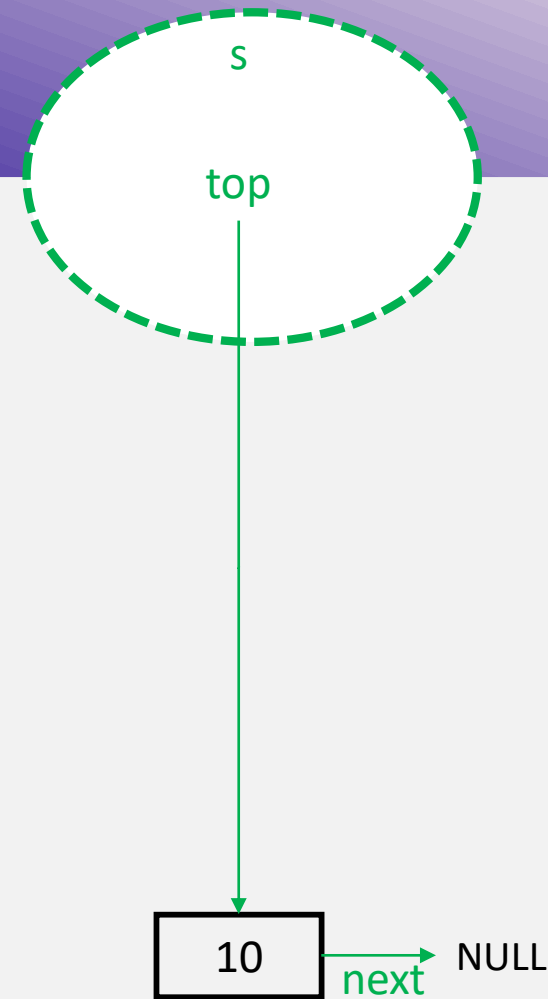


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```

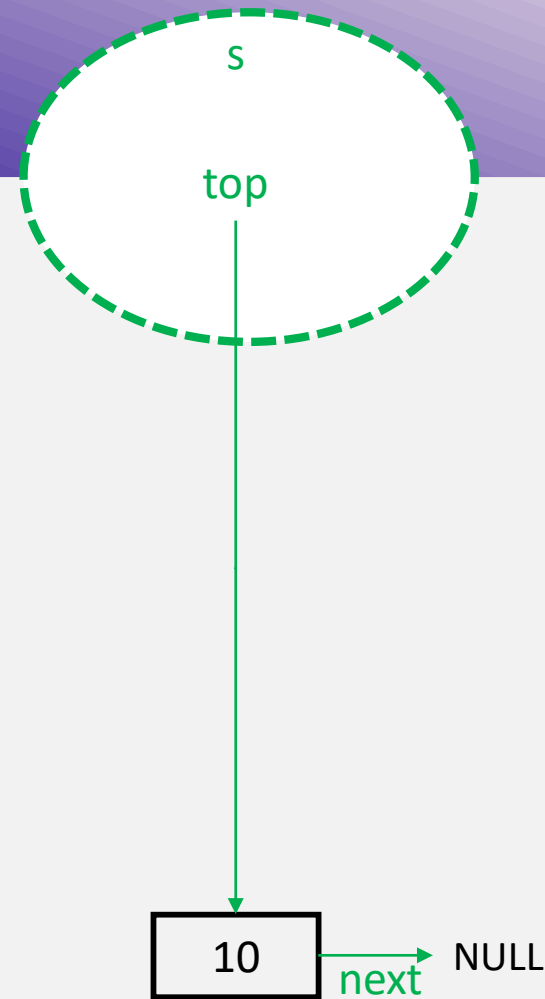


Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```



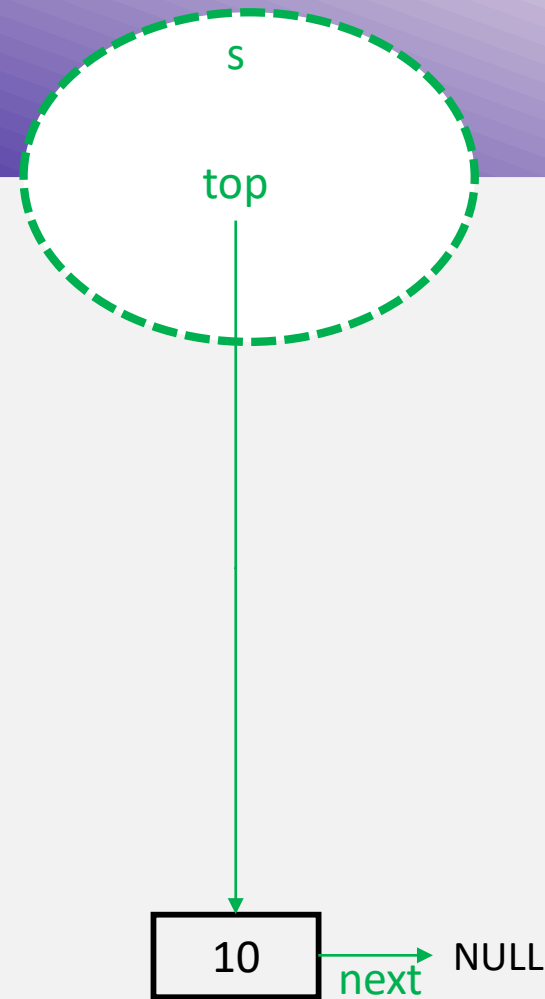
Pilhas

◦ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```



```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```



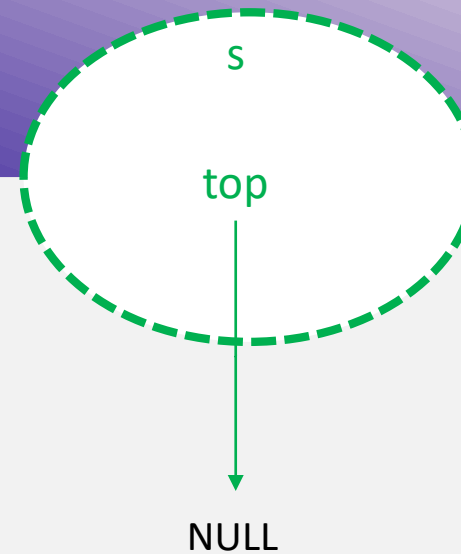
```
Stack (top -> bottom): 10 -> NULL
```

Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```



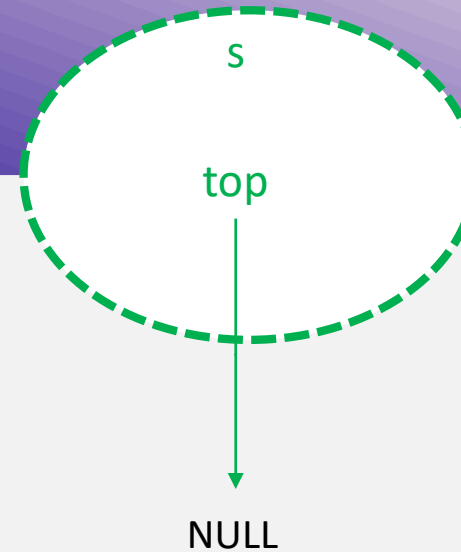
```
Stack (top -> bottom): 10 -> NULL
```

Pilhas

○ IMPLEMENTAÇÃO DE PILHA

```
62 // Example
63 int main() {
64     Stack* s = (Stack*) malloc(sizeof(Stack));
65     s->top = NULL;
66
67     s = push(s, 10);
68     s = push(s, 20);
69     s = push(s, 30);
70
71     s = pop(s);
72     s = pop(s);
73
74     print_stack(s);
75
76     free_stack(s);
77     return 0;
78 }
```

```
31 // Pop
32 Stack* pop(Stack* s) {
33     if (s->top == NULL) return s;
34     Node* temp = s->top;
35     s->top = s->top->next;
36     free(temp);
37     return s;
38 }
```



```
Stack (top -> bottom): 10 -> NULL

...Program finished with exit code 0
Press ENTER to exit console.
```